

Test Case Process(テストケース作成の手順)

「カテゴリXのカバレッジが必要」という状態から、`forensic-agent-tests` 内の 稼働中・監査済みのケースになるまでの、段階を追った手順です。以下の関連 ドキュメントと合わせて読んでください(これらの代わりではありません)。

- `TEST_OBJECTIVES.md` — カテゴリの定義、シナリオ例、カテゴリごとの関連 スキル/データソース。
- `TEST_CASE_MATRIX.md` — 現在何がカバーされていて、何がカバーされていないか。
- `SOURCES.md` — 証拠データを供給・生成できるものとその状態。
- `../AGENTS.md` — 下記のPhase 2～3で参照する、EvidenceForge固有の「生成 → 移植」の具体的な手順。

このドキュメントは順番通りのチェックリストであり、上記の各資料は其中で 参照する資料です。ここに内容を重複して書かないでください — もし他の資料の 段落をこのファイルにコピーしうになったら、代わりにリンクしてください。

この手順自体も今後変わる前提です。実際のケースで使う前に草案として書かれた ものなので、これを使って最初に作るケースは、ケース自体の検証であると同時に この手順の検証でもあると捉え、そこで出てきた課題をもとにこのドキュメントを 改訂してください。

Phase 0 — 提案(次に進む前にユーザーとClaude双方の承認が必要)

このフェーズが承認されるまで、作成・生成・ファイル作成は一切行いません。これは形式的な手続きではなく、厳格なゲートです。設計ミス、ケースを半分 作ってしまった後ではなく、1段落の議論で済むうちに見つけるためのものです。

以下のテンプレートを使って提案を作成し、Phase 1に進む前に明確な承認を 得てください。

提案テンプレート

Case Proposal: <作業用スラッグ名>

****対象カテゴリ****(`TEST_OBJECTIVES.md` より、`TEST_CASE_MATRIX.md` の Coverage Gaps を踏まえて選定): <例: 6 + 7>

****ストーリーの概要****(2～4文 - 環境、登場人物、何が起きるか):

****攻撃かどうか?*** 攻撃の場合: 判別対象となる無害な類似パターン (同じアカウント/ホスト/パターンを共有する無害な事象) はあるか?

****データソース****(`SOURCES.md` より – 「Adopted(採用済み)」である必要があります。「Candidate(候補)」を使用/昇格させる提案の場合は明示してください)：

****設計の参考にしたスキル****(試験問題/採点基準の設計に使う `Anthropic-Cybersecurity-Skills` のエントリ、攻撃側のリアリティが必要な場合は `transilienceai/communitytools` のエントリ – いずれもAUT向けではなく、作成時の参考資料としてのみ使用)：

****試験問題の想定範囲****(設問数、何を問うか – 最終的な文言ではない)：

****事前に把握しているリスク****(既存の `KNOWN_DEFICIENCIES.md` に記載された、再発しうるエンジンの制約や、このストーリーで自己矛盾が起きそうな点など)：

- ☐ ユーザーの承認済み

Phase 1 — データソース自身の環境で作成・調整する

EvidenceForge(現時点で唯一の Adopted 済み生成ツール)の場合:

`/Users/mlcs/Documents/github/EvidenceForge` の中だけで作業し、このリポジトリ 中では作業しません。具体的なコマンドと環境構築については `../AGENTS.md` の Phase 1 を参照してください。

他のデータソースの場合: まだ手順が存在しないため、このフェーズの一部として 新たに手順を書いてください。EvidenceForgeの手順がそのまま流用できると 仮定しないでください。

`eforge validate` (または他のデータソースにおける同等のコマンド)が通り、 提案内容に照らしてシナリオが物語として完結するまでここで調整を繰り返します。

Phase 2 — 何かを移植する前に、生データの整合性を確認する

このステップは、`single-host-linux-rce` の `apache2 / nginx` の不一致や 因果関係の逆転バグを、公開前に発見できたはずのステップです。Phase 3の前に 必ず行ってください。後回しにしないでください。

- 試験問題が依拠する予定のすべての事実(アイデンティティ、プロセスの親子 関係、タイムスタンプ、バイト数、相関ID)について、`GROUND_TRUTH.md` からではなく、実際にレンダリングされたデータから直接確認してください。`GROUND_TRUTH.md` 自体が、実際にレンダリングされた内容と食い違っている ことがあります(`rdp-remote-file-write` の約21分のタイミングのずれを参照)。
- 内部矛盾がないか確認してください: プロセスの親子関係は宣言された `services` と一致しているか? 同一の因果関係の連鎖について、独立して タイムスタンプが付与された複数のソース間でイベントの順序は整合しているか? `ENVIRONMENT.md` で計画している内容と矛盾する点はないか?

- 問題があり、シナリオ作成側で修正可能であれば、修正して再生成してください。作成レベルでは修正できないと(想定ではなく実際に試した上で)確認できた場合は、それはPhase 6で判断すべきことであり、ここで取り繕うべきではありません。

Phase 3 — 移植する

`../AGENTS.md` のPhase 2のチェックリストをすべて実施してください。

1. 生成元の入力ファイル(`scenario.yaml` のみ)を `forensic-agent-answers/generators/evidenceforge/<slug>/` にコピーし、正確なソースのバージョン/コミット/シード値を記録したREADMEを添えます。これは `forensic-agent-tests/` ではなく `forensic-agent-answers/` に置きます — `scenario.yaml` は実質的にYAML形式のケースの正解データだからです。
2. 出力を分割します: 安全な証拠データ + `ENVIRONMENT.md` → `cases/<slug>/data/`、解答が明らかになる補助ファイル → `forensic-agent-answers/case-<slug>/supporting/`、調査上の価値がなく実在のツールを特定できてしまうリスクのある生成ツールの管理用ファイルは → 完全に除外します。
3. 何か他のファイルを書く前に、`data/` だけでなく `cases/<slug>/` ツリー全体を監査してください。
 - 生成ツールの名称/ベンダー名/ブランド名を大文字小文字を区別せずgrepし — `README.md` / `CHANGELOG.md` を含め、AUTが読める場所に1件もヒットしないことを確認します。
 - ストーリーや解答が漏れるパターン(例: `"storyline_id"`、`"kind": *"red_herring"`、EvidenceForgeの場合は `"activity":` など — データソースごとにパターンを調整)をgrepしてください。補助ファイルに書かれた「用途」の説明を鵜呑みにせず、実際にレンダリングされた内容を確認してください。
4. ケース一式を作成します: `README.md`、`AGENTS.md` (薄いルーター+証拠の索引。出典情報は含めない)、`TASK.md`、`EXAM.md` (設問のみ)、`CHANGELOG.md`、`.gitignore` (`QUESTION_ANSWERS.md` を含める)。
5. 解答一式を作成します: `BRIEFING.md`、`AGENTS.md` (採点者向けの指示)、`grading_schema.md` (`EXAM.md` と1対1対応)。

Phase 4 — 実際のアナリストの手順に沿って試験問題を設計する

対象カテゴリごとに、`TEST_OBJECTIVES.md` の「関連スキル」列 (Anthropic-Cybersecurity-Skills)を参照し、そのカテゴリの一般論ではなく、実際のアナリストの業務で確認される内容を問う設問にしてください。攻撃シナリオの場合は、ストーリー自体を

`transilienceai/communitytools` の攻撃側スキルと照合し、現実的な手口(偵察 → 侵害 → 目的達成)になっているか確認してください。「攻撃っぽく見えるか」ではありません。

Phase 5 — 各設問を個別に生データと突き合わせて確認する

Phase 2とは別の作業です。Phase 2はデータ自体の整合性を確認するものですが、こちらは作成した各設問について、回答可能か、曖昧さがないか、偶然の一致(たとえば、たまたまストーリーと同じ識別子を持つ通常のノイズ)に依拠して いないかを確認するものです。設問がこの確認に通らず、かつテスト内容を変えずに言い換えることもできない場合は、その設問は削除してください — 無理に試験に含めて、採点基準側で問題を取り繕うことはしないでください。

Phase 6 — 稼働開始前の独立監査

ケースを作成したのとは別のコンテキスト — 続きのセッションではなく新規 セッション — からレビューを受けてください。これは単なる形式的な確認ではありません。今回のセッションで行った外部監査では、同じコンテキストでの 自己レビューでは見逃していた実際のバグ(因果関係の逆転、プロセスの親子 関係の不一致、誤検知を招くアイデンティティのバグ)が見つかりました。これは正式な1ステップとして予算・時間を確保してください。単なる儀礼的な 確認ではありません。

監査で見つかった内容には、以下の「残す/修正する/廃棄する」の基準を 適用してください。

- **設問の採点が依拠する事実についてデータ自体が自己矛盾している** → ケースを廃棄するか保留する(このリポジトリの `_discarded/case-single-host-linux-rce/` にあるパターンを参照 — 削除ではなく移動し、理由を書き残してください。廃棄されたケースは、単に非アクティブと印を付けるのではなく、証拠データ・タスク側の一式を `forensic-agent-tests` から完全に削除します — アクティブに使われなくなった時点で、AUT向けのリポジトリに残す理由がなくなるためです。監査の 理由を含めた記録全体はこちらのリポジトリに残します)。
- **文言の曖昧さが原因で、データ自体には問題がない** → 言い換える。
- **本物の癖ではあるが採点対象の事実には影響しない** → `BRIEFING.md` / `grading_schema.md` に記録した上でそのまま残す。

Phase 7 — 管理用ドキュメントの更新

- `TEST_CASE_MATRIX.md` — 新しい行を追加し、カテゴリごとに実際にテスト された箇所にXを付ける(より厳しい定義に従うこと: ある設問がそのカテゴリ 固有の調査手法を実際に必要としている場合のみXを付け、単に一般的に 関連しているだけでは付けない)。
- `TEST_OBJECTIVES.md` — このケースによってそこに記載されたギャップが 解消・部分的に解消された場合は更新する。
- `SOURCES.md` — このケースによって、あるデータソースの `Status` が Candidateから Adoptedに昇格した場合は更新する。